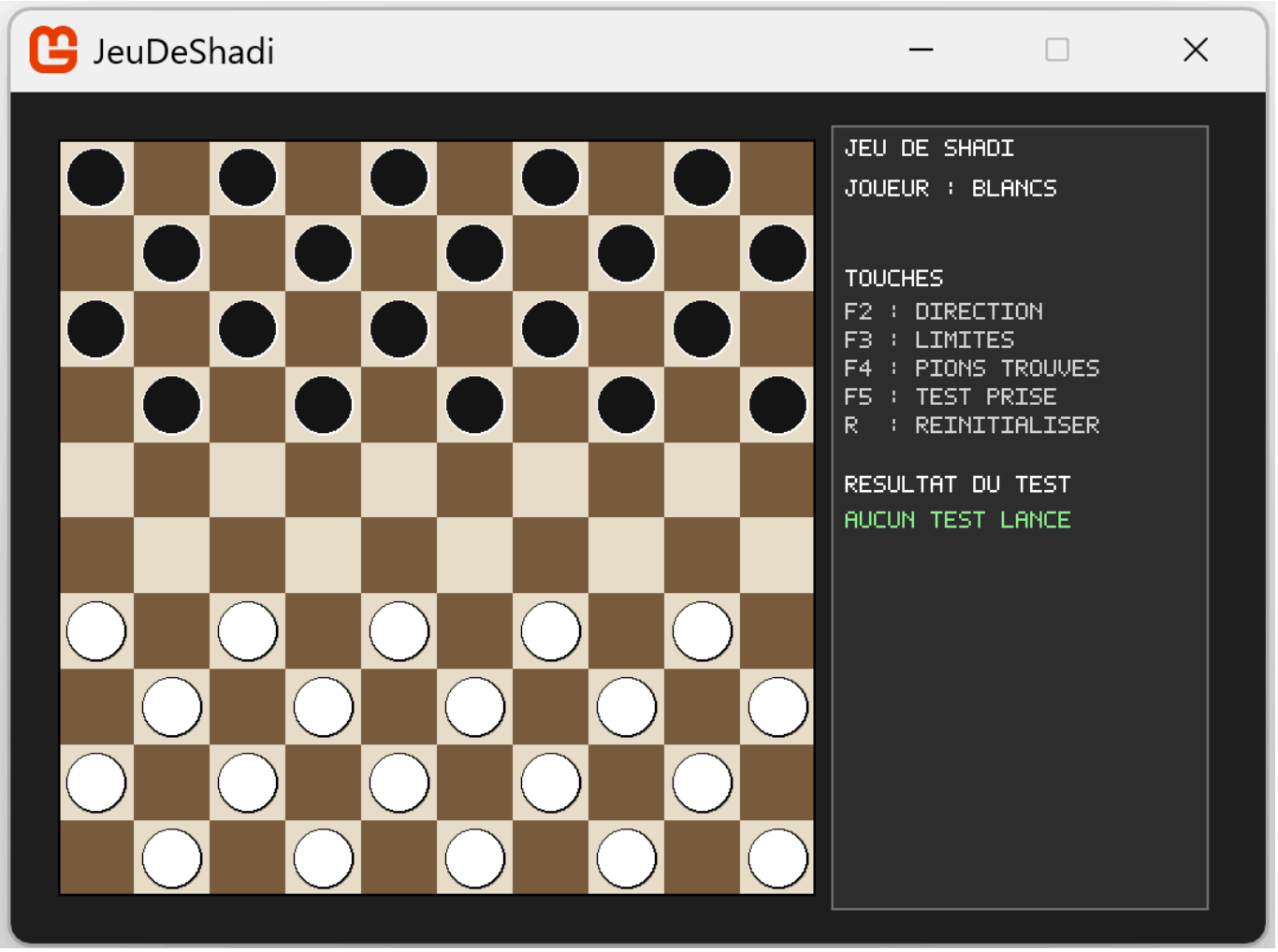
Projet C# — Jeu de Shadi



Le jeu de Shadi est un jeu de plateau stratégique et proche du jeu de Dames.

Règles du jeu de Shadi :

- Les joueurs jouent chacun à leur tour. Les blancs commencent.
- Chaque pion peut se déplacer d'une case vers l'avant en diagonale.
- Un pion peut en prendre un autre en sautant par dessus un pion adverse.
- La prise peut également s'effectuer en arrière.
- La prise est obligatoire mais non multiple (une seule prise à la fois).
- Si un des joueurs est dans l'impossibilité de jouer il perd la partie.
- Le joueur qui parvient le premier à atteindre la dernière rangée avec un pion est vainqueur.

Piste bleue — parcours guidé

1. Objectif de la piste bleue

Vous allez programmer progressivement la logique du **Jeu de Shadi**.

L'objectif principal de ce projet est de consolider les notions de programmation que vous connaissez déjà :

- variables
- conditions
- boucles
- tableaux
- méthodes et fonctions

Le projet utilise également des éléments que vous n'avez pas encore étudiés, notamment **MonoGame**, les listes et quelques éléments de programmation orientée objet.

Vous n'avez pas à apprendre ces notions pour réussir la piste bleue.

Chaque fois qu'une partie du programme nécessite une notion qui n'a pas encore été étudiée :

- le code est déjà fourni ou
- une méthode d'aide vous est donnée ou
- l'appel exact à effectuer vous est indiqué

Votre travail consiste donc à vous concentrer sur **la logique du jeu**.

La piste bleue est volontairement progressive.

Le niveau d'accompagnement évolue au cours du projet :

- **étapes 1 et 2 : très guidées** ;
- **étapes 3 et 4 : guidées** ;
- **à partir de l'étape 5 : autonomie accompagnée**.

Même lorsque le guidage diminue, les méthodes et outils mis à votre disposition sont toujours présentés et expliqués. En revanche, vous devez progressivement construire vous-même l'algorithme nécessaire pour atteindre l'objectif demandé.

2. Préparer le projet sous Windows

Le projet vous est fourni sous la forme d'un fichier ZIP.

Windows peut considérer un fichier téléchargé comme provenant d'un autre ordinateur et bloquer certains fichiers contenus dans le projet. Cela peut empêcher MonoGame de fonctionner correctement.

2.1 Débloquer le ZIP avant de l'extraire

Avant d'extraire le projet :

1. faites un clic droit sur le fichier ZIP
2. choisissez **Propriétés**
3. si l'option **Débloquer** est affichée en bas de la fenêtre, cochez-la
4. cliquez sur **Appliquer**
5. extrayez ensuite le fichier ZIP.

> Il est préférable de débloquer le ZIP **avant** de l'extraire.

2.2 Premier lancement

Lancez ensuite le projet.

Au début du travail, vous devez déjà voir apparaître :

- une fenêtre
- un plateau de 10 × 10 cases.

Il est normal que le jeu ne fonctionne pas encore complètement : vous allez programmer sa logique progressivement.

---

3. Technologie utilisée pour l'affichage

Le projet est écrit en **C#** et utilise **.NET 9**.

L'affichage graphique est réalisé avec **MonoGame**.

MonoGame s'occupe notamment de :

- créer la fenêtre
- dessiner le plateau
- dessiner les pions
- lire la souris
- détecter les clics
- afficher les cases sélectionnées
- rafraîchir l'écran

Vous n'avez pas à programmer ces éléments.

---

4. Les données principales du jeu

Deux variables sont particulièrement importantes pour votre travail.

4.1 `plateauDeJeu`

Le plateau est représenté par un tableau à deux dimensions :

```
int[,] plateauDeJeu = new int[10, 10];
```

Vous pouvez l'imaginer comme une grille de 10 colonnes et 10 lignes.

Une case se lit ainsi :

```
plateauDeJeu[x, y]
```

avec :

- `x` = colonne
- `y` = ligne.

La case `(0, 0)` se trouve en haut à gauche.

4.2 Valeur contenue dans une case

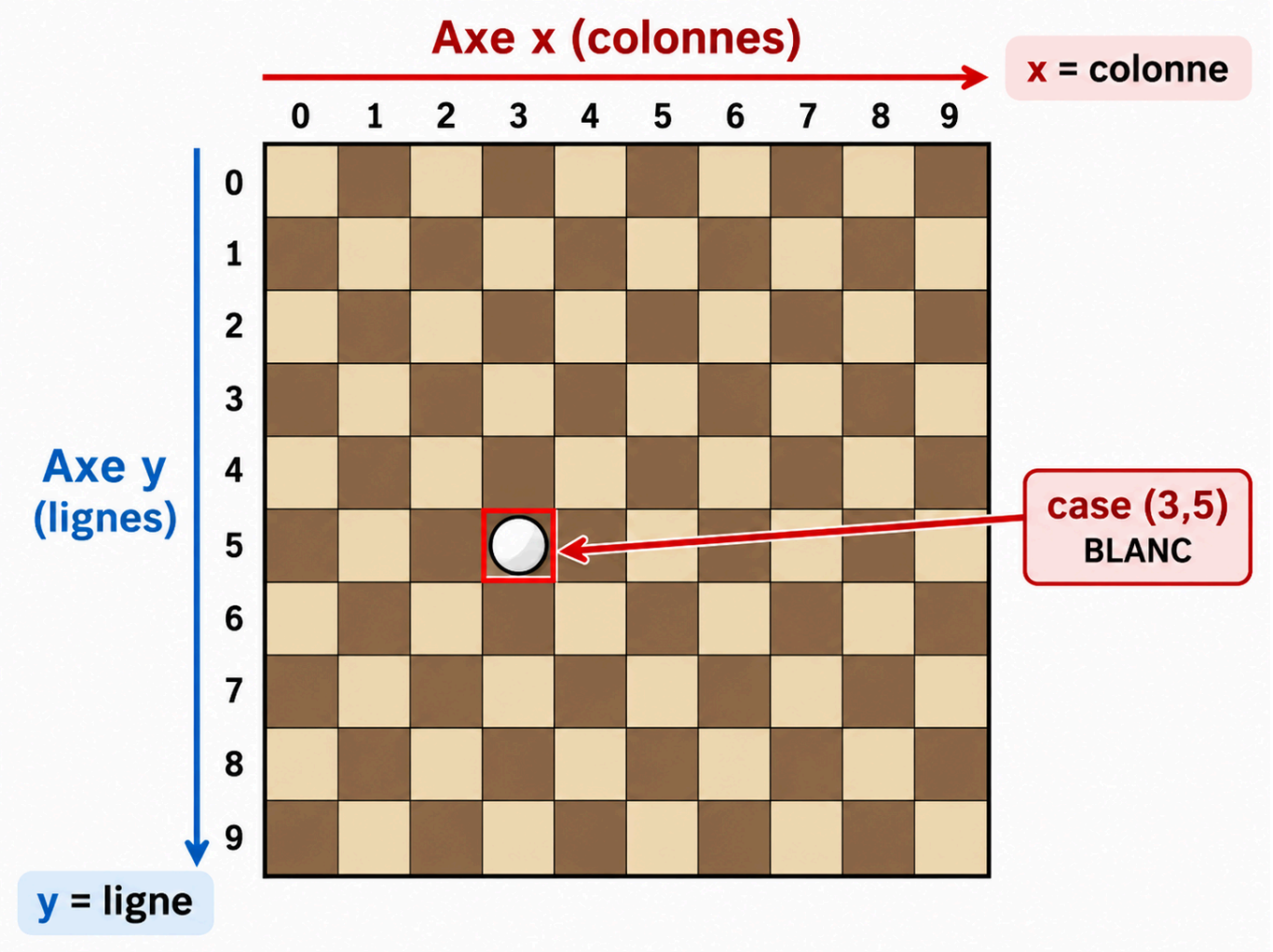
Trois constantes sont fournies :

```
const int VIDE = 0;
const int BLANC = 1;
const int NOIR = 2;
```

Ainsi :

```
plateauDeJeu[3, 5] = BLANC;
```

signifie qu'un pion blanc est placé sur la case `(3, 5)`.



Pour vider cette case :

```
plateauDeJeu[3, 5] = VIDE;
```

4.3 `joueurCourant`

La variable :

```
joueurCourant
```

contient la couleur du joueur qui doit jouer.

Sa valeur est donc :

```
BLANC
```

ou :

```
NOIR
```

Cette variable est fournie et accessible aux méthodes que vous allez compléter.

---

5. Comment l'affichage fonctionne avec votre code

Vous n'avez pas besoin de demander directement à MonoGame de dessiner un pion.

Le principe est le suivant :

```
vous modifiez plateauDeJeu
            ↓
le code graphique lit plateauDeJeu
            ↓
MonoGame dessine le nouvel état du plateau
```

Par exemple, si votre code exécute :

```
plateauDeJeu[4, 6] = BLANC;
```

le code graphique fourni dessine automatiquement un pion blanc à cette position.

C'est très important : **vous programmez les données et les règles, pas l'affichage**.

---

6. Comment la souris fonctionne

La gestion de la souris est également déjà programmée.

Le code fourni :

1. détecte la case sur laquelle le joueur clique
2. vérifie s'il s'agit d'un pion du joueur courant
3. demande à votre code quels déplacements sont possibles
4. affiche les destinations
5. détecte la destination choisie
6. appelle votre méthode qui applique le coup

Vous n'avez donc aucune instruction MonoGame à écrire pour la souris.

---

7. Les notions plus avancées sont masquées

Le programme utilise en interne une liste de coups et une structure appelée `Coup`.

Vous n'avez pas encore étudié ces notions.

Vous n'avez donc pas à écrire du code comme :

```
new Coup(...)
```

ou :

```
coups.Add(...)
```

À la place, vous utiliserez des méthodes d'aide fournies.

Pour enregistrer un déplacement normal :

```
AjouterDeplacement(xDepart, yDepart, xArrivee, yArrivee);
```

Pour enregistrer une prise :

```
AjouterPrise(xDepart, yDepart, xArrivee, yArrivee);
```

Le code fourni s'occupe du reste.

---

8. Cycle général d'un tour

Le fonctionnement d'un tour peut être résumé ainsi :

```
le joueur clique sur un pion
        ↓
le programme recherche les coups possibles
        ↓
les destinations sont affichées
        ↓
le joueur clique sur une destination
        ↓
votre code modifie plateauDeJeu
        ↓
MonoGame redessine automatiquement le plateau
        ↓
le joueur suivant peut jouer
```

---

9. Où devez-vous travailler ?

Dans `Game1.cs`, cherchez :

```
// ZONE ÉLÈVE — PISTE BLEUE
```

Vous devez principalement modifier les méthodes contenues dans cette zone.

Ne modifiez pas les parties marquées :

```
// MÉTHODES D'AIDE FOURNIES — NE PAS MODIFIER
```

et :

```
// CODE FOURNI : AFFICHAGE ET INTERACTIONS
```

---

10. Les outils de test fournis

Pour la piste bleue, le projet contient plusieurs outils de vérification déjà programmés.

Vous n'avez pas besoin de comprendre le code MonoGame qui réalise ces tests. Vous devez simplement utiliser les touches indiquées.

| Touche | Vérification |

|---|---|

| `R` | Recharger la position initiale |

Le panneau d'aide et de test

La fenêtre du jeu contient maintenant deux zones :

```
+----------------------------+----------------------+
|                            |                      |
|      PLATEAU DE JEU        |  PANNEAU D'AIDE      |
|                            |  ET DE TEST          |
|                            |                      |
+----------------------------+----------------------+
```

Le panneau de droite est entièrement fourni. Vous ne devez pas programmer son affichage.

Il vous sert à vérifier immédiatement votre travail. Lorsqu'un test est lancé, vous devez comparer les valeurs affichées avec les valeurs attendues.

| `F2` | Tester votre méthode `DirectionAvant()` |

| `F3` | Tester votre méthode `DansLesLimites()` |

| `F4` | Afficher ou masquer le test visuel de l'étape 4 |

| `F5` | Charger une situation simple permettant de tester une prise |

Lorsque vous utilisez `F2`, `F3`, `F4` ou `F5`, regardez le **panneau situé à droite du plateau**.

Ce panneau affiche :

- le joueur courant
- les touches de test disponibles
- le résultat détaillé du dernier test.

Vous n'avez rien à programmer pour afficher ce panneau : il fait partie du code fourni.

Le but de ces outils est simple : **à la fin de chaque étape, vous devez pouvoir vérifier immédiatement votre travail**.

---

11. Étape 1 — Initialiser le plateau

Niveau d'accompagnement : très guidé

Dans cette étape, le raisonnement est largement détaillé. Les notions à utiliser et la manière générale de construire la solution vous sont indiquées.

Méthode à compléter :

```
void InitialiserPlateau()
```

Votre objectif est de placer les pions dans leur position de départ.

Règles :

- le plateau possède 10 lignes et 10 colonnes
- seules les cases où `(x + y)` est impair sont utilisées
- les Noirs occupent les lignes `0` à `3`
- les Blancs occupent les lignes `6` à `9`.

Vous devrez utiliser deux boucles imbriquées :

```
for (...)
{
    for (...)
    {
        ...
    }
}
```

Pour savoir si une case est jouable, vous pouvez utiliser le modulo `%` :

```
(x + y) % 2
```

Vous devrez ensuite affecter `NOIR` ou `BLANC` à la bonne case de `plateauDeJeu`.

Méthodes fournies à utiliser

Pour cette première étape, **aucune méthode d'aide n'est nécessaire**.

Vous travaillez directement avec :

```
plateauDeJeu[x, y]
```

et avec les constantes :

```
VIDE
BLANC
NOIR
```

C'est volontaire : cette première étape doit vous permettre de revoir uniquement les boucles, les conditions et les tableaux.

Ce que vous devez voir

Lancez le programme, ou appuyez sur `R` si le programme est déjà lancé.

Vous devez voir :

- **20 pions noirs** dans les quatre rangées du haut
- **20 pions blancs** dans les quatre rangées du bas
- les deux rangées centrales (`4` et `5`) vides
- les pions uniquement sur les cases jouables du damier.

Les Blancs se trouvent donc en bas et **les Blancs commencent la partie**.

Ne continuez pas tant que le placement initial n'est pas correct.

---

12. Étape 2 — Déterminer la direction d'un pion

Niveau d'accompagnement : très guidé

Dans cette étape, le raisonnement est presque entièrement donné. Votre travail consiste principalement à le traduire correctement en C#.

Méthode à compléter :

```
int DirectionAvant(int couleur)
```

Les Blancs avancent vers le haut du tableau :

```
-1
```

Les Noirs avancent vers le bas :

```
+1
```

Vous devez utiliser une condition et `return`.

Exemple de raisonnement :

```
SI la couleur est BLANC
    retourner -1
SINON
    retourner 1
```

Méthodes fournies à utiliser

Pour cette étape, **aucune méthode d'aide n'est nécessaire**.

La valeur reçue dans le paramètre :

```
couleur
```

sera soit `BLANC`, soit `NOIR`.

Votre méthode doit uniquement retourner la valeur correspondant au sens de déplacement.

Cette méthode sera ensuite réutilisée plus tard par :

```
GenererCoupsPourPion(...)
```

Elle permet d'éviter de répéter partout la logique qui détermine le sens de déplacement des Blancs et des Noirs.

Ce que vous devez voir

Lancez le programme puis appuyez sur `F2`.

Le panneau doit afficher quelque chose de proche de :

```
TEST ETAPE 2

BLANC : -1  ATTENDU : -1

NOIR  : +1  ATTENDU : +1

RESULTAT : OK
```

Si `A CORRIGER` apparaît, vérifiez votre `if`, votre `else` et vos `return`.

---

13. Étape 3 — Vérifier les limites du plateau

Niveau d'accompagnement : guidé

L'objectif et les contraintes sont clairement indiqués, mais vous devez construire vous-même l'expression logique qui permet de vérifier les coordonnées.

Méthode à compléter :

```
bool DansLesLimites(int x, int y)
```

Cette méthode doit retourner `true` uniquement si la position existe dans le tableau.

Pour un plateau de 10 × 10 :

```
x doit être compris entre 0 et 9

y doit être compris entre 0 et 9
```

Vous devez vérifier les quatre conditions.

Vous pouvez utiliser :

```
&&
```

pour signifier **ET**.

Méthodes fournies à utiliser

Pour cette étape, **aucune autre méthode n'est nécessaire**.

Vous devez simplement utiliser les paramètres :

```
x

y
```

et la constante :

```
TailleGrille
```

Cette méthode sera ensuite très importante dans la recherche des déplacements et des prises.

Son rôle est de protéger le programme contre un accès à une case qui n'existe pas.

Par exemple, avant d'utiliser :

```
plateauDeJeu[xArrivee, yArrivee]
```

vous pourrez vérifier :

```
DansLesLimites(xArrivee, yArrivee)
```

Ce que vous devez voir

Lancez le programme puis appuyez sur F3.

Le panneau doit confirmer les quatre essais :

```
TEST ETAPE 3
(0,0)   ATTENDU VRAI : OK
(9,9)   ATTENDU VRAI : OK
(-1,3)  ATTENDU FAUX : OK
(10,4)  ATTENDU FAUX : OK
RESULTAT : OK
```

Si une ligne indique `ERREUR`, corrigez votre méthode avant de poursuivre.

---

14. Étape 4 — Chercher les pions du joueur courant

Niveau d'accompagnement : guidé

Les opérations à réaliser et les méthodes disponibles sont présentées. Vous devez organiser ces éléments correctement dans votre code.

Méthode à compléter :

```
void RecalculerCoupsLegaux()
```

Deux appels sont déjà fournis :

```
CommencerRechercheCoups();
```

et :

```
TerminerRechercheCoups();
```

Vous ne devez pas modifier ces appels.

Entre les deux, vous devez :

1. parcourir tout `plateauDeJeu`
2. vérifier si la case contient un pion de `joueurCourant`
3. si oui, appeler la méthode de test fournie
4. puis demander la recherche des coups pour ce pion.

Méthodes fournies à utiliser

`CommencerRechercheCoups()`

```
CommencerRechercheCoups();
```

Cette méthode prépare le programme à une nouvelle recherche de coups.

Elle remet à zéro les informations de la recherche précédente.

Vous devez l'appeler **avant** de parcourir le plateau.

`MarquerPionTrouvePourTest(x, y)`

```
MarquerPionTrouvePourTest(x, y);
```

Cette méthode sert uniquement au test visuel de l'étape.

Elle permet de surligner un pion trouvé par votre boucle.

Paramètres :

- `x` : colonne du pion trouvé
- `y` : ligne du pion trouvé.

Elle ne modifie pas les règles du jeu.

`GenererCoupsPourPion(x, y)`

```
GenererCoupsPourPion(x, y);
```

Cette méthode devra rechercher les déplacements possibles pour le pion situé en `(x, y)`.

À cette étape, elle n'est peut-être pas encore terminée, mais vous devez déjà l'appeler pour chaque pion du joueur courant.

Paramètres :

- `x` : colonne du pion
- `y` : ligne du pion.

`TerminerRechercheCoups()`

```
TerminerRechercheCoups();
```

Cette méthode termine la recherche et applique les règles générales liées aux coups trouvés, notamment la priorité des prises.

Vous devez l'appeler **après** avoir parcouru tout le plateau.

Ce que vous devez voir

1. Appuyez sur `R`.
2. Appuyez sur `F4`.
3. Regardez le plateau et le panneau de droite.

Au début de la partie, les **20 pions blancs** doivent être surlignés.

Le panneau doit notamment afficher :

```
TEST ETAPE 4

PIONS TROUVES : 20

ATTENDU AU DEPART : 20
```

Aucun pion noir ne doit être surligné.

---

15. Étape 5 — Rechercher les coups possibles d'un pion

Niveau d'accompagnement : autonomie accompagnée

À partir de cette étape, les règles à respecter, les méthodes disponibles et le résultat attendu sont expliqués, mais l'algorithme n'est plus fourni. Vous devez déterminer vous-même comment organiser vos boucles, conditions et calculs.

Méthode concernée :

```
void GenererCoupsPourPion(int x, int y)
```

Cette méthode doit répondre à la question :

> **Sur quelles cases le pion situé en `(x, y)` peut-il jouer ?**

Pour un pion donné, deux types de coups sont possibles :

1. un **déplacement simple**
2. une **prise**.

Le début de la méthode est déjà fourni :

```
int couleurPion = plateauDeJeu[x, y];
int avant = DirectionAvant(couleurPion);
```

La variable `couleurPion` contient la couleur du pion.

La variable `avant` contient sa direction normale d'avancement.

Le code fournit également :

```
int[] decalagesX = { -1, 1 };
```

Ce tableau représente les deux directions horizontales possibles :

```
-1 -> gauche

+1 -> droite
```

À partir de cette étape, vous travaillez en **autonomie accompagnée**.

Les méthodes nécessaires sont expliquées, mais vous devez choisir vous-même comment les combiner pour construire votre solution.

Méthodes fournies ou déjà construites à utiliser

`DirectionAvant(couleur)`

```
int avant = DirectionAvant(couleurPion);
```

Vous avez construit cette méthode à l'étape 2.

Elle vous évite de devoir écrire de nouveau une condition pour savoir si le pion monte ou descend.

`DansLesLimites(x, y)`

```
DansLesLimites(xArrivee, yArrivee)
```

Vous avez construit cette méthode à l'étape 3.

Utilisez-la **avant** de lire une case calculée du tableau.

Elle permet d'éviter un accès comme :

```
plateauDeJeu[-1, 4]
```

qui provoquerait une erreur.

`AjouterDeplacement(...)`

```
AjouterDeplacement(xDepart, yDepart, xArrivee, yArrivee);
```

Cette méthode est fournie.

Elle enregistre un déplacement simple que votre code vient de détecter.

Vous n'avez pas à savoir comment la liste des coups est gérée en interne.

Vous devez simplement l'appeler lorsque vous êtes certain que le déplacement est autorisé.

`AjouterPrise(...)`

```
AjouterPrise(xDepart, yDepart, xArrivee, yArrivee);
```

Cette méthode est fournie.

Elle enregistre une prise possible.

Comme pour `AjouterDeplacement(...)`, la gestion interne de la liste est déjà programmée.

`EstUnAdversaire(x, y, couleurPion)`

```
EstUnAdversaire(xMilieu, yMilieu, couleurPion)
```

Cette méthode est fournie.

Elle permet de savoir si la case indiquée contient bien un pion de la couleur opposée.

Elle retourne un booléen :

```
true  -> la case contient un adversaire

false -> ce n'est pas un adversaire
```

Vous l'utiliserez surtout dans la recherche des prises.

16. Étape 5A — Chercher les déplacements simples

Niveau d'accompagnement : autonomie accompagnée

Les méthodes utiles et les conditions à respecter sont expliquées. À vous de construire l'algorithme permettant de trouver les déplacements simples.

Un déplacement simple consiste à avancer d'**une case en diagonale vers l'avant**.

Prenons un pion blanc situé en :

```
(4, 6)
```

Comme les Blancs montent dans le tableau, les deux destinations possibles sont :

```
(3, 5)    et    (5, 5)
```

Pour chacune des deux diagonales, vous devez :

1. calculer la colonne d'arrivée
2. calculer la ligne d'arrivée
3. vérifier que la position existe
4. vérifier que la case est vide
5. si tout est correct, enregistrer le déplacement.

Méthodes fournies ou déjà construites à utiliser

`DirectionAvant(couleur)`

Déjà utilisée au début de `GenererCoupsPourPion(...)`.

Elle vous donne la valeur à ajouter à `y` pour avancer dans le bon sens.

`DansLesLimites(x, y)`

Utilisez-la avant d'accéder à la destination calculée.

Exemple :

```
if (DansLesLimites(xArrivee, yArrivee))
{
    ...
}
```

`AjouterDeplacement(...)`

Utilisez-la seulement lorsque vous avez vérifié que la destination est valide et vide.

Elle reçoit :

```
AjouterDeplacement(
    xDepart,
    yDepart,
    xArrivee,
    yArrivee);
```

Dans cette méthode :

- `xDepart` et `yDepart` correspondent ici à `x` et `y`
- `xArrivee` et `yArrivee` sont les coordonnées que vous venez de calculer.

À vous de construire l'algorithme

Vous connaissez maintenant :

- les deux directions horizontales possibles ;
- le sens d'avancement du pion ;
- la méthode permettant de vérifier les limites ;
- la manière d'enregistrer un déplacement valide.

Avant de coder, réfléchissez aux questions suivantes :

- quelles coordonnées devez-vous calculer pour chaque diagonale ?
- dans quel ordre devez-vous effectuer les vérifications ?
- à quel moment pouvez-vous appeler `AjouterDeplacement(...)` ?

Construisez ensuite votre solution avec les boucles et conditions que vous jugez nécessaires.

Ce que vous devez voir

1. Appuyez sur R.
2. Cliquez sur un pion blanc situé sur la rangée avant des Blancs.

Les destinations autorisées doivent apparaître graphiquement.

Un pion loin du bord peut avoir deux destinations.

Un pion près d'un bord peut n'en avoir qu'une.

---

17. Étape 5B — Chercher les prises

Niveau d'accompagnement : autonomie accompagnée

Les règles de prise et les méthodes disponibles sont expliquées. À vous de déterminer les calculs, les boucles et l'ordre des vérifications.

Une prise est plus complexe qu'un déplacement simple.

Cette fois, le pion saute par-dessus un pion adverse.

Vous devez donc examiner :

1. la case intermédiaire
2. la case d'arrivée.

Pour une prise, le Jeu de Shadi autorise un saut vers l'avant **ou vers l'arrière**.

Le code fournit :

```
int[] directionsY = { avant, -avant };
```

Vous devez donc combiner :

```
gauche / droite
```

avec :

```
avant / arrière
```

Méthodes fournies ou déjà construites à utiliser

`DansLesLimites(x, y)`

Vous devez vérifier les coordonnées avant d'utiliser le tableau.

Pour une prise, vous devrez vérifier au minimum :

- la case intermédiaire
- la case d'arrivée.

`EstUnAdversaire(x, y, couleurPion)`

Cette méthode est particulièrement importante ici.

Elle vous permet de vérifier si la case intermédiaire contient bien un adversaire.

Exemple :

```
EstUnAdversaire(xMilieu, yMilieu, couleurPion)
```

Vous n'avez donc pas besoin de programmer vous-même la comparaison entre Blancs et Noirs.

`AjouterPrise(...)`

Lorsque vous avez vérifié :

- qu'un adversaire se trouve au milieu
- que la destination est libre
- que les coordonnées sont valides

appelez :

```
AjouterPrise(x, y, xArrivee, yArrivee);
```

Le code fourni se chargera de mémoriser cette prise.

À vous de construire l'algorithme

À partir de cette étape, aucun pseudo-code complet n'est fourni.

Avant de programmer, notez dans votre journal de travail :

- les coordonnées dont vous avez besoin
- les boucles nécessaires
- l'ordre des vérifications
- les conditions qui doivent être vraies pour autoriser une prise.

Ce que vous devez voir

1. Appuyez sur `F5`.
2. Un pion blanc est placé en `(3,6)` et un pion noir en `(4,5)`.
3. Cliquez sur le pion blanc.

La destination :

```
(5,4)
```

doit être proposée.

Si elle n'apparaît pas, vérifiez une seule chose à la fois.

---

18. La prise obligatoire est déjà gérée

Niveau d'accompagnement : autonomie accompagnée

Cette partie explique un mécanisme fourni par le projet. Vous devez comprendre son rôle afin de savoir comment vos propres méthodes s'intègrent au fonctionnement général.

Le Jeu de Shadi impose une prise lorsqu'au moins une capture est disponible.

Cette règle utilise une liste de coups interne au projet.

Comme les listes ne font pas partie des objectifs de cette piste, cette partie est déjà programmée.

Méthodes fournies concernées

`CommencerRechercheCoups()`

Elle prépare une nouvelle recherche de coups.

`AjouterDeplacement(...)`

Elle enregistre les déplacements simples trouvés.

`AjouterPrise(...)`

Elle enregistre les prises trouvées.

`TerminerRechercheCoups()`

Cette méthode analyse les coups enregistrés.

Si au moins une prise existe, elle conserve uniquement les prises et ignore les déplacements simples.

Vous n'avez donc pas à écrire vous-même la règle de priorité.

Votre responsabilité est uniquement de détecter correctement tous les coups avant l'appel à `TerminerRechercheCoups()`.

---

19. Étape 6 — Appliquer un coup

Niveau d'accompagnement : autonomie accompagnée

Les objectifs et les méthodes disponibles sont indiqués, mais vous devez construire vous-même l'ordre des traitements et les conditions nécessaires.

Méthode à compléter :

```
void AppliquerCoup(
    int xDepart,
    int yDepart,
    int xArrivee,
    int yArrivee,
    bool estUnePrise)
```

Vous savez maintenant trouver les coups possibles.

Il reste à appliquer réellement le coup choisi par le joueur.

La partie graphique vous transmet déjà :

- `xDepart`
- `yDepart`
- `xArrivee`
- `yArrivee`
- `estUnePrise`.

Vous devez maintenant organiser vous-même le traitement.

Votre méthode doit :

- conserver la couleur du pion
- vider la case de départ
- placer le pion sur la case d'arrivée
- supprimer le pion adverse en cas de prise
- détecter une éventuelle victoire
- sinon terminer le tour.

Méthodes fournies à utiliser

`DeclarerVictoire(couleurPion)`

```
DeclarerVictoire(couleurPion);
```

Cette méthode est fournie.

Utilisez-la lorsque vous avez déterminé que le pion vient d'atteindre la rangée de victoire.

Elle s'occupe de la fin de partie et de l'affichage correspondant.

`TerminerTour(couleurPion)`

```
TerminerTour(couleurPion);
```

Cette méthode est fournie.

Utilisez-la lorsqu'il n'y a pas de victoire.

Elle s'occupe notamment de :

- changer de joueur
- recalculer les coups possibles pour le joueur suivant
- vérifier si ce joueur peut encore jouer.

Vous ne devez donc pas programmer vous-même le changement de joueur à cet endroit.

Méthodes déjà construites utiles

Pour cette étape, vous n'avez normalement pas besoin d'appeler directement :

```
DansLesLimites(...)

DirectionAvant(...)
```

car le coup transmis à `AppliquerCoup(...)` a déjà été validé par la recherche des coups.

À vous de construire l'algorithme

Aucun pseudo-code n'est fourni pour cette étape.

Vous devez déterminer :

- dans quel ordre modifier le tableau
- à quel moment mémoriser la couleur du pion
- comment retrouver la case intermédiaire lors d'une prise
- à quel moment appeler `DeclarerVictoire(...)` ou `TerminerTour(...)`.

Ce que vous devez voir

Déplacement normal

- le pion quitte la case de départ
- il apparaît sur la case d'arrivée
- le joueur change.

Prise

Avec `F5` :

- le pion blanc part de `(3,6)`
- il arrive en `(5,4)`
- le pion noir en `(4,5)` disparaît.

Victoire

Lorsqu'un pion atteint la rangée adverse, la victoire doit être détectée.

---

20. Étape 7 — Validation autonome du jeu

Niveau d'accompagnement : autonomie accompagnée

Aucune solution algorithmique supplémentaire n'est donnée. Vous devez utiliser les outils de test et vos connaissances du programme pour identifier et corriger les erreurs.

À cette étape, aucune aide algorithmique supplémentaire n'est fournie.

Votre objectif est de vérifier que votre jeu respecte réellement toutes les règles.

Méthodes fournies à utiliser

Aucune nouvelle méthode n'est introduite.

Vous devez maintenant être capable d'utiliser seul les méthodes déjà rencontrées :

```
DirectionAvant(...)

DansLesLimites(...)

AjouterDeplacement(...)

AjouterPrise(...)

EstUnAdversaire(...)

DeclarerVictoire(...)

TerminerTour(...)
```

L'objectif est justement de montrer que vous comprenez désormais à quoi elles servent et dans quelles situations les utiliser.

Tests à réaliser

Testez au minimum :

- un déplacement blanc vers la gauche
- un déplacement blanc vers la droite
- un déplacement noir vers la gauche
- un déplacement noir vers la droite
- un pion situé au bord du plateau
- une prise vers l'avant
- une prise vers l'arrière
- une prise obligatoire
- une victoire des Blancs
- une victoire des Noirs
- un joueur sans aucun coup possible.

Pour chaque erreur :

1. reproduisez-la
2. identifiez la méthode concernée
3. corrigez une seule chose à la fois
4. retestez.

---

21. Étape 8 — Relire et améliorer votre code

Niveau d'accompagnement : autonomie accompagnée

Vous devez analyser votre propre code, être capable de le justifier et améliorer les parties qui vous semblent inutilement complexes.

Le jeu fonctionne maintenant.

Relisez uniquement le code que vous avez écrit.

Méthodes fournies à utiliser

Aucune nouvelle méthode n'est nécessaire.

Cette étape consiste à analyser votre propre code.

Vérifiez notamment :

- que vos noms de variables sont compréhensibles
- que vos conditions ne sont pas inutilement compliquées

- que vos boucles sont nécessaires
- que vous savez expliquer chaque appel à une méthode fournie
- que vous savez expliquer pourquoi vous utilisez `DansLesLimites(...)` avant certains accès au tableau
- que vous savez expliquer la différence entre `AjouterDeplacement(...)` et `AjouterPrise(...)`
- que vous savez expliquer le rôle de `TerminerTour(...)`

Si vous ne savez pas expliquer une partie de votre propre code, essayez de la simplifier.

L'objectif final de la piste bleue est que vous terminiez le projet avec plus d'autonomie que lorsque vous l'avez commencé.